

Express js

Express setup: `npm i express`

1.Web server and Page handling with 404 error

```
const express = require('express')
const app = express ()
const path = require('path')
const PORT = process.env.PORT || 3500

app.get('^/$|index(.html)?' , (req , res) =>
{
  // res.send('Hi Prasanth')
  res.sendFile(path.join(__dirname,'index.html'))
})

app.get('/new-page(.html)?' , (req , res) =>
{
  // res.send('Hi Prasanth')
  res.sendFile(path.join(__dirname,'new-page.html'))
})

app.get('/old-page(.html)?' , (req , res) =>
{
  res.redirect(301 , 'new-page.html')
})

app.get('/hello' , (req,res,next) =>
{
  console.log('Loading Hello...')
  next()
}, (req,res) =>
{
  res.send('Hi hello')
})

app.get('/*' , (req,res) =>
{
  res.status(404).send('404 page not found !')
})

app.listen( PORT ,() => console.log('Server is running on',PORT))
```

2.Chain Functions

```
const express = require('express')
const app = express ()
const path = require('path')
const PORT = process.env.PORT || 3500

app.get('^/$|index(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'index.html'))
})

app.get('/new(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'new-page.html'))
})

app.get('/hello' , (req,res,next) =>
{
    console.log('Loading Hello ')
    next()
},(req,res) =>
{
    res.send('Hello ')
})

const one = (req,res,next) =>
{
    console.log('One')
    next()
}
const two = (req,res,next) =>
{
    console.log('Two')
    next()
}
const three = (req,res) =>
{
    console.log('three')
    res.send('Finished')
}

app.get('/chain' , [one,two,three])

app.get('/*', (req,res) =>
{
    res.status(404).send('404 Page not found')
```

```

    })

app.listen( PORT , () =>
{
    console.log('Server running on ',PORT)
})

```

3.Middleware

```

const express = require('express')
const app = express ()
const path = require('path')
const PORT = process.env.PORT || 3500

app.use(express.urlencoded({extended:false}))
app.use(express.json())
app.use(express.static(path.join(__dirname, '/public')))

app.get('^/$|index(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname, 'index.html'))
})

app.get('/new(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname, 'new-page.html'))
})

app.get('/hello' , (req,res,next) =>
{
    console.log('Loading Hello ')
    next()
},(req,res) =>
{
    res.send('Hello ')
})

const one = (req,res,next) =>
{
    console.log('One')
    next()
}

const two = (req,res,next) =>
{
    console.log('Two')
    next()
}

```

```

}
const three = (req,res) =>
{
  console.log('three')
  res.send('Finished')
}

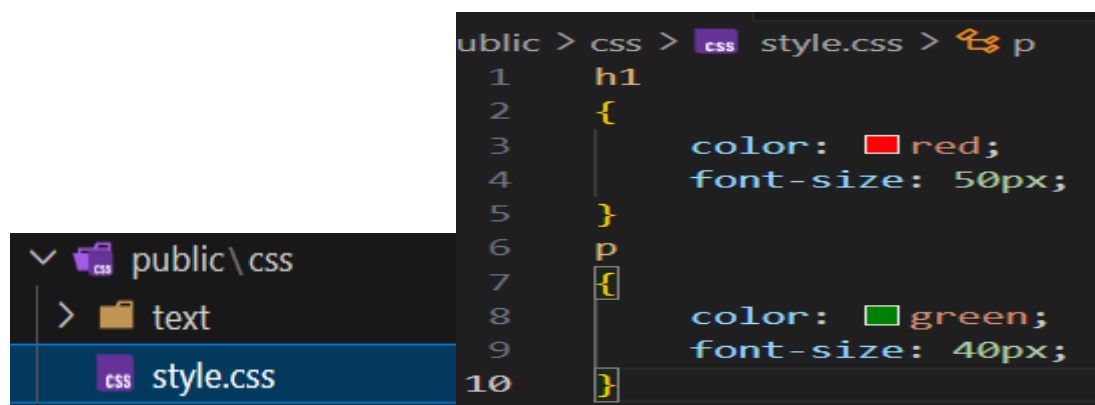
app.get('/chain' , [one,two,three])

app.get('/*', (req,res) =>
{
  res.status(404).send('404 Page not found')
})

app.listen( PORT , () =>
{
  console.log('Server running on ',PORT)
})

```

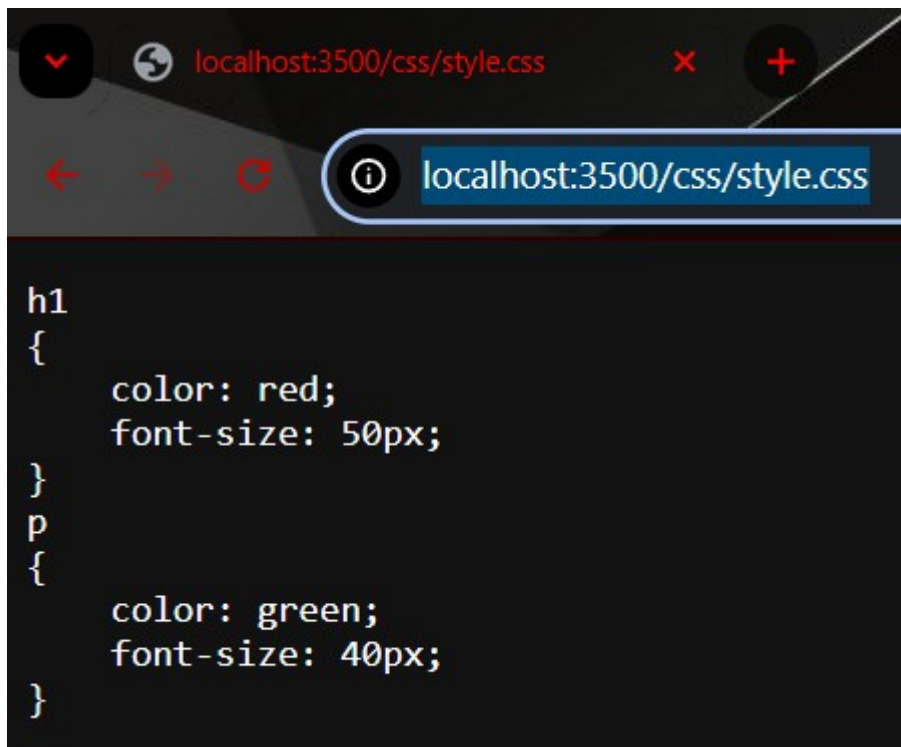
OUTPUT:



Prasanth

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Consequuntur quasi reiciendis provident aliquid delectus. Commodi saepe pariatur temporibus ex illo?

```
app.use(express.json())
```



Method and Path :

```
const express = require('express')
const path = require('path')
const app = express ()
const PORT = process.env.PORT || 3500

//Ena ena files lam use panniruku nu paakurathuku use:
app.use( (req,res,next) => // next ah use panna adhu middle ware
{
  console.log( `${req.method} | ${req.path}` )
  next()
})

app.use(express.urlencoded({extended:false}))
app.use(express.json())

app.use(express.static(path.join(__dirname,'public'))) //File Access
(important Line)

app.get('^/$|index(.html)?' , (req,res) =>
{
```

```

    res.sendFile(path.join(__dirname, 'index.html'))
  })

  app.get('/new-page(.html)?' , (req,res) =>
  {
    res.sendFile(path.join(__dirname, 'new-page.html'))
  })

  app.listen( PORT ,() =>
  {
    console.log('Server is running on',PORT)
  })

  app.get('/*', (req,res) =>
  {
    res.status(404).send('404 Page not found')
  })

```

OUTPUT:

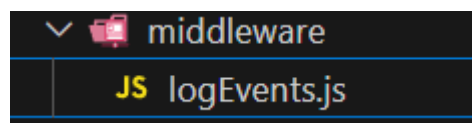
```

Server is running on 3500
GET | /
GET | /css/style.css
GET | /images/red%20car.png

```

Middleware and Logs:

Step 1: Create the path and the logEvents file



logEvents.js:

```

const { format } = require('date-fns')
const { v4 : uuid} = require('uuid')

const fs = require('fs')
const fsPromises = require('fs').promises
const path = require('path')

const logEvents = async(message,filename) =>
{
  const datetime = format(new Date(), 'dd/MM/yyyy\thH:mm:ss')

```

```

const logItem = `${datetime}\t${uuid()}\t${message}\n`

try
{
    if(!fs.existsSync(path.join(__dirname,'Logs')))
    {
        await fsPromises.mkdir(path.join(__dirname,'Logs'))
        await fsPromises.appendFile(path.join(__dirname,'Logs',filename),'
Date          Time          uID          Message\n')
    }
    await fsPromises.appendFile(path.join(__dirname,'Logs',filename),
logItem)
}
catch(err)
{
    console.error(err)
}
}

module.exports = logEvents

```

web_server.js:

```

const express = require('express')
const logEvents = require('./middleware/logEvents')
const path = require('path')
const app = express ()
const PORT = process.env.PORT || 3500
//Ena ena files lam use panniruku nu paakurathuku use:
app.use( (req,res,next) => //use next as a middleware
{
    logEvents(`${req.method}|${req.url}`, 'reqlogs.txt')
    next()
}))

app.use(express.urlencoded({extended:false}))
app.use(express.json())

app.use(express.static(path.join(__dirname,'public'))) //File Access
(important Line)

app.get('^/$|index(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'index.html'))
}))

```

```

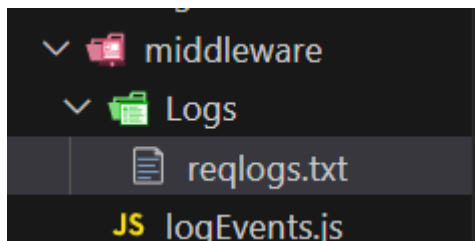
app.get('/new-page(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'new-page.html'))
})

app.listen( PORT ,() =>
{
    console.log('Server is running on',PORT)
})

app.get('/*', (req,res) =>
{
    res.status(404).send('404 Page not found')
})

```

After Refresh::::::::::::::::::



Reqlogs.txt

```

middleware > Logs > reqlogs.txt
1  |   Date      Time      uID      Message
2  | 03/03/2025  20:28:05  f9775bba-b192-4ce9-8048-7c981ebce992  GET|/
3  | 03/03/2025  20:28:05  1eb93230-810e-444a-bee1-3a327be2867e  GET|/css/style.css
4  | 03/03/2025  20:28:05  75c322f6-6f39-4a78-b6dc-237688e2492c  GET|/images/red%20car.png

```

Middleware and Logger:

logEvents.js

```

const { format } = require('date-fns')
const { v4 : uuid} = require('uuid')

const fs = require('fs')
const fsPromises = require('fs').promises
const path = require('path')

const logEvents = async(message,filename) =>
{
    const datetime = format(new Date(), 'dd/MM/yyyy\thH:mm:ss')
    const logItem = `${datetime}\t${uuid()}\t${message}\n`

```

```

    try
    {
        if(!fs.existsSync(path.join(__dirname,'Logs')))
        {
            await fsPromises.mkdir(path.join(__dirname,'Logs'))
            await fsPromises.appendFile(path.join(__dirname,'Logs',filename),'
Date      Time      uID      Message\n')
        }
        await fsPromises.appendFile(path.join(__dirname,'Logs',filename),
logItem)
    }
    catch(err)
    {
        console.error(err)
    }
}

const logger = (req,res,next) => //use next as a middleware
{
    logEvents(`${req.method}|${req.url}`, 'reqlogs.txt')
    next()
}

module.exports = {logEvents,logger}

```

web_server.js

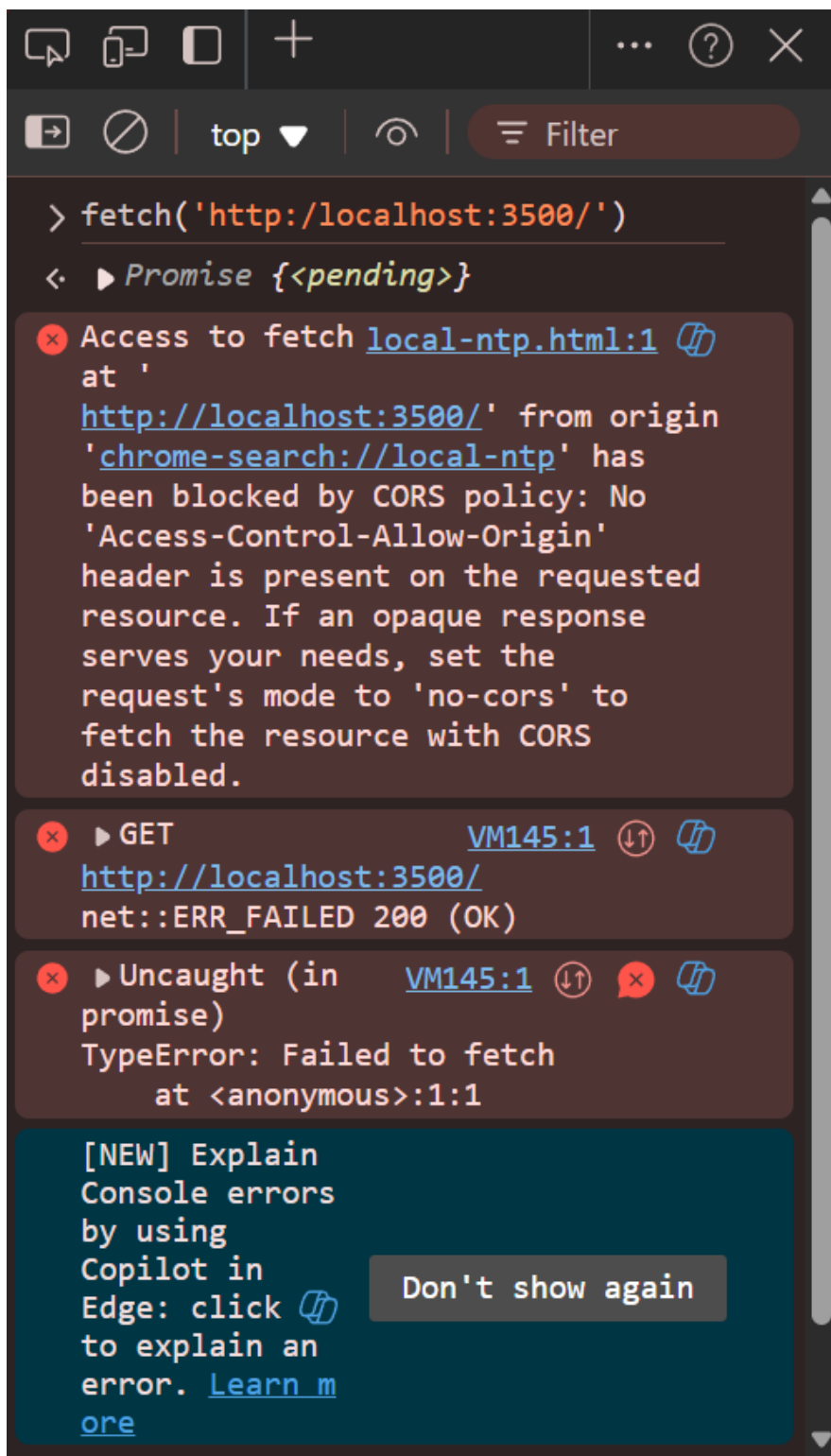
```

//=====Middle ware + Logger=====
const express = require('express')
const {logger} = require('./middleware/logEvents')
const path = require('path')
const app = express ()
const PORT = process.env.PORT || 3500

//Ena ena files lam use panniruku nu paakurathuku use:
app.use(logger)

```

Third-party- Middle ware:



To **avoid** this error: install cors:

```
npm i cors
```

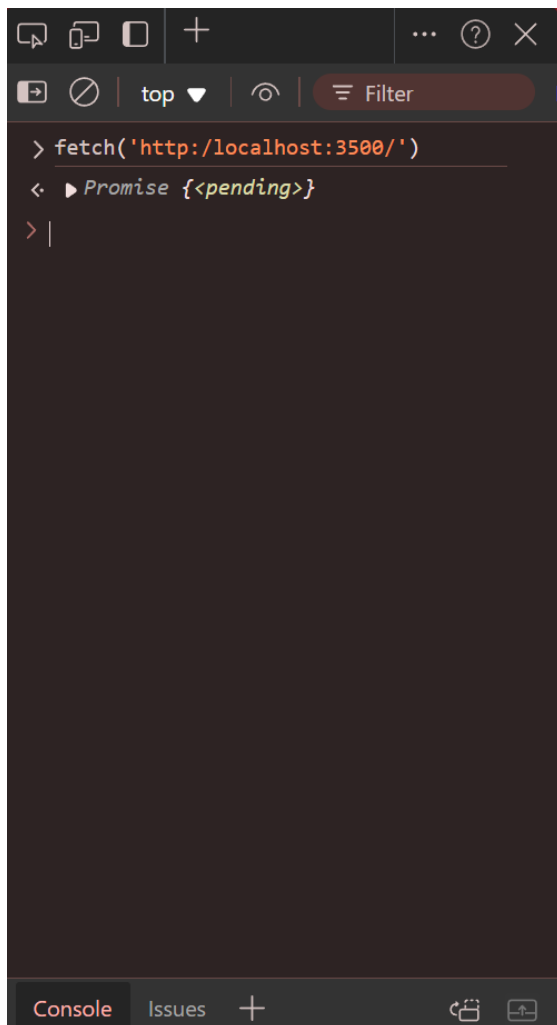
web_server.js

//=====Middle ware + Logger + cors=====

```
const express = require('express')
const {logger} = require('./middleware/logEvents')
const cors = require('cors')
const path = require('path')
const app = express ()
const PORT = process.env.PORT || 3500

//Ena ena files lam use panniruku nu paakurathuku use:
app.use(logger)
app.use(cors())
app.use(express.urlencoded({extended:false}))
app.use(express.json())
```

After using the cors in our code:



Cors Access and Deny:

web_server.js

```
//=====Cors Policy Access and Deny=====

const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')

//Custom Middleware
app.use(logger)

//Cors access giving and declining
const whitelist =
['https://www.google.com','http://127.0.0.1:5500','http://localhost:3500']
const corsOptions = {
  origin : (origin,callback) =>
  {
    if(whitelist.includes(origin))
    {
      callback(null,true)
    }
    else
    {
      callback(new Error('Blocked due to cors policy'))
    }
  },
  optionsSuccessStatus : 200
}

app.use(cors(corsOptions))

app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname,'/public')))

app.get('^/$|/index(.html)?' , (req,res) =>
{
  res.sendFile(path.join(__dirname,'index.html'))
})

app.get('/new' , (req,res) =>
{
  res.sendFile(path.join(__dirname,'new-page.html')
  )
})
```

```

}))

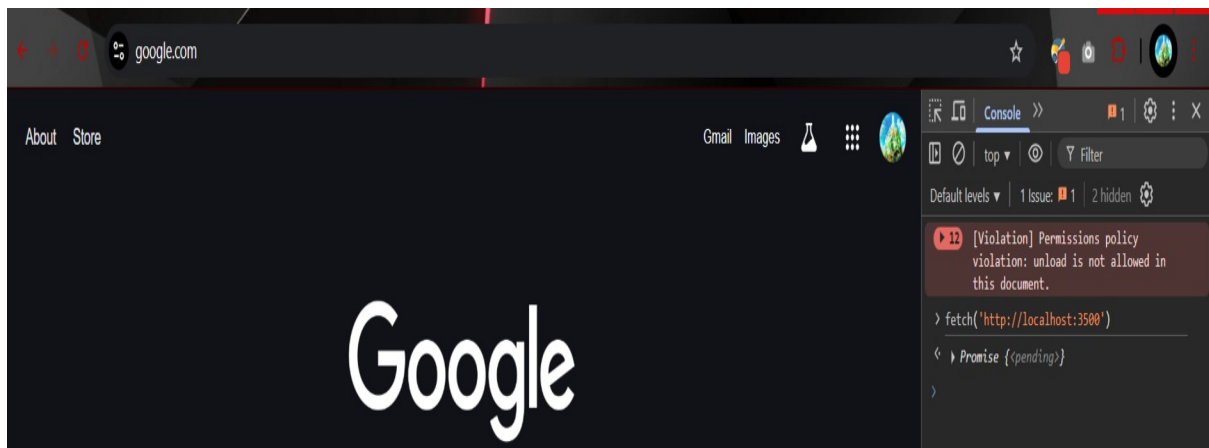
app.get('/hello' , (req,res) =>
{
    res.send('Hello...')
})

app.listen( PORT , () =>
{
    console.log('Server is running on',PORT)
})

app.get('/*' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'404.html'))
})

```

OUTPUT:



To Solve this Clearly ::

web_server.js:

```

const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')
//Custom Middleware
app.use(logger)

//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500','http://localhost:3500']

```

```

const corsOptions = {
  origin : (origin,callback) =>
  {
    if(whitelist.includes(origin) || !origin)//!origin -production apa thevaila
    {
      callback(null,true)
    }
    else
    {
      callback(new Error('Blocked Due to Cross Policy !'))
    }
  },
  optionsSuccessStatus : 200
}
app.use(cors(corsOptions))
app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname,'/public')))
app.get('^/$|/index(.html)?' , (req,res) =>
{
  res.sendFile(path.join(__dirname,'index.html'))
})
app.get('/new' , (req,res) =>
{
  res.sendFile(path.join(__dirname,'new-page.html')
)
})
app.get('/hello' , (req,res) =>
{
  res.send('Hello...')
})
app.get('/*' , (req,res) =>
{
  res.sendFile(path.join(__dirname,'404.html'))
})
app.use(errorHandler)

app.listen( PORT , () =>
{
  console.log('Server is running on',PORT)
})

```

```

>  Logs
  JS errorHandler.js
  JS logEvents.js

```

errorHandler.js

```
const {logEvents} = require('./logEvents')

const errorHandler = (err,req,res,next) =>
{
  logEvents(`${err.name} ${err.message}`, 'errLog.txt')
  res.status(500).send(err.message)
  next()
}

module.exports = errorHandler
```

logEvents.js:

```
const { format } = require('date-fns')
const { v4 : uuid } = require('uuid')
const fs = require('fs')
const fsPromises = require('fs').promises
const path = require('path')
const logEvents = async(message,file_name) =>
{
  const datetime = format(new Date(), 'dd/MM/yyyy\thH:mm:ss')
  const logItem = `${datetime}\t${uuid()}\t${message}\n`
  try
  {
    if(!fs.existsSync(path.join(__dirname,'..','Logs')))
    {
      await fsPromises.mkdir(path.join(__dirname,'..','Logs'))
      await
      fsPromises.appendFile(path.join(__dirname,'..','Logs',file_name), '
      Date
      Time          uID          Method  Message      Path\n')
      await
      fsPromises.appendFile(path.join(__dirname,'..','Logs',file_name), logItem)
    }
    catch(err){
      console.error(err)
    }
  }
}

const logger = (req,res,next) =>
{
  logEvents(`${req.method}\t${req.headers.origin}\t${req.url}`, 'req_log.txt')
  console.log(`${req.method} ${req.path}`)
  next()
}

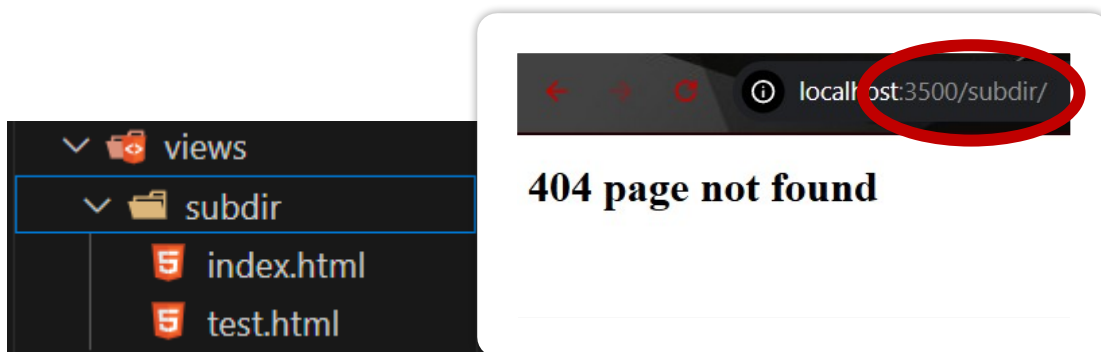
module.exports = {logger,logEvents}
```

Middlewere all :

```
app.all('*', (req,res) =>
{
  if(req.accepts('html'))
  {
    res.sendFile(path.join(__dirname,'404.html'))
  }
  else if(req.accepts('json'))
  {
    res.json({'error':'404 json error'})
  }
  else
  {
    res.type('txt').send('404 type error')
  }
})
```

Routing:

Oru sub directories la irukura files ku epadi access kudukurathu ?
adhan routing la learn panna porom.



Showing wrong file when searching the files in the sub directory folder.

To Solve this follow the steps

Step 1: Create a Routes Folder and make a file called subdir.js

Routes>subdir.js:

```
const express = require('express')
const router = express.Router()
const path = require('path')

router.get( '^/$|index(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'..','views','subdir','index.html'))
})

router.get( '/test(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname,'..','views','subdir','test.html'))
})

module.exports = router
```

Step 2: use the router app in web_server.js:

```
const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')

//Custom Middleware
app.use(logger)

//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500','http://localhost:3500']
const corsOptions = {
    origin : (origin,callback) =>
    {
        if(whitelist.includes(origin) || !origin)
        {
            callback(null,true)
        }
        else
        {
            callback(new Error('Blocked Due to Cross Policy !'))
        }
    },
    optionsSuccessStatus : 200
}
```

```
}
app.use(cors(corsOptions))

app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname, '/public')))
app.use('/subdir', require('./Routes/subdir'))
app.get('^/$|/index(.html)?' , (req,res) =>
{
    res.sendFile(path.join(__dirname, 'views', 'index.html'))
})

app.get('/new' , (req,res) =>
{
    res.sendFile(path.join(__dirname, 'views', 'new-page.html')
    )
})

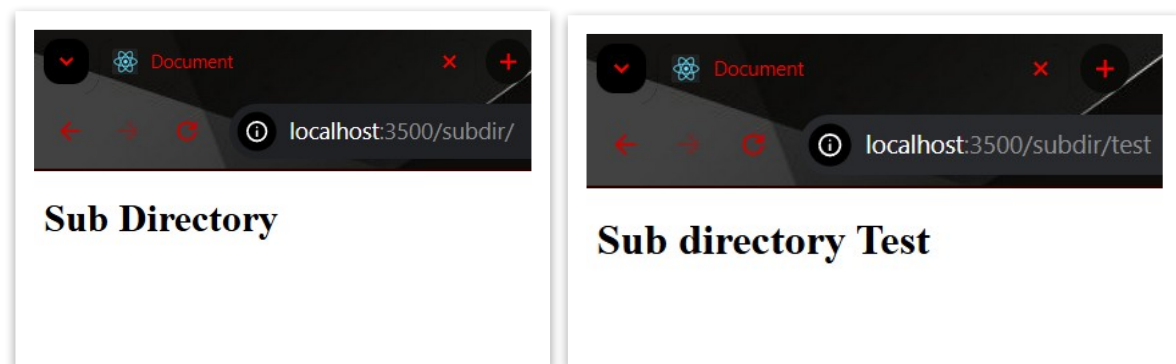
app.get('/hello' , (req,res) =>
{
    res.send('Hello...')
})

app.all('*', (req,res) =>
{
    if(req.accepts('html'))
    {
        res.sendFile(path.join(__dirname, 'views', '404.html'))
    }
    else if(req.accepts('json'))
    {
        res.json({'error':'404 json error'})
    }
    else
    {
        res.type('txt').send('404 type error')
    }
})

app.use(errorHandler)

app.listen( PORT , () =>
{
    console.log('Server is running on',PORT)
})
```

Result:



Another try:: (important)

web_server.js:

```
//=====Make Another root=====
const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')

//Custom Middleware
app.use(logger)

//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500','http://localhost:3500']
const corsOptions = {
  origin : (origin,callback) =>
  {
    if(whitelist.includes(origin) || !origin)
    {
      callback(null,true)
    }
    else
    {
      callback(new Error('Blocked Due to Cross Policy !'))
    }
  },
  optionsSuccessStatus : 200
}
```

```

}
app.use(cors(corsOptions))

app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname, '/public')))

app.use('/subdir', require('./Routes/subdir'))

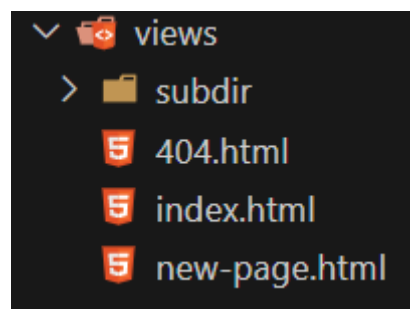
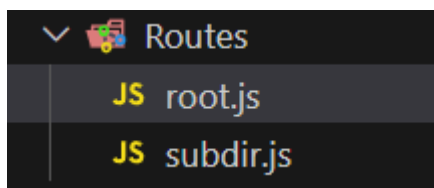
app.use('/', require('./Routes/root'))

app.use(errorHandler)

app.listen( PORT , () =>
{
  console.log('Server is running on',PORT)
})

```

Routes/root.js:



```

const express = require('express')
const router = express.Router()
const path = require('path')

router.get('^/$|/index(.html)?' , (req,res) =>
{
  res.sendFile(path.join(__dirname, '..', 'views', 'index.html'))
})

router.get('/new' , (req,res) =>
{
  res.sendFile(path.join(__dirname, '..', 'views', 'new-page.html'))
})

router.get('/hello' , (req,res) =>
{
  res.send('Hello...')
})

```

```

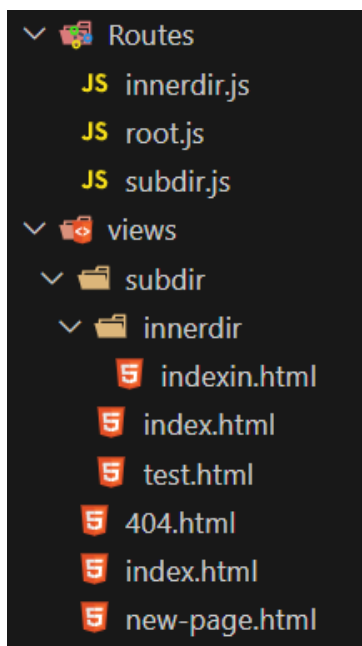
router.all('*', (req,res) =>
  {
    if(req.accepts('html'))
    {
      res.sendFile(path.join(__dirname,'..','views','404.html'))
    }
    else if(req.accepts('json'))
    {
      res.json({'error':'404 json error'})
    }
    else
    {
      res.type('txt').send('404 type error')
    }
  })

module.exports = router

```

Adhigapresengithanam successfully done:

web_server.js:



1. **GET:** This method allows for the server to find the data you requested and sends it back to you.
2. **PUT:** If you perform the 'PUT' request, then the server will update an entry in the database.
3. **POST:** This method permits the server to create a new entry in the database.
4. **DELETE:** This method allows the server to delete an entry in the database.

```

//=====Make Another root adhukulla another
innerdir=====
const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')

//Custom Middleware
app.use(logger)

//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500','http://localhost:3500']
const corsOptions = {
  origin : (origin,callback) =>
  {
    if(whitelist.includes(origin) || !origin)
    {
      callback(null,true)
    }
    else
    {
      callback(new Error('Blocked Due to Cross Policy !'))
    }
  },
  optionsSuccessStatus : 200
}
app.use(cors(corsOptions))

app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname, '/public')))

app.use('/subdir',express.static(path.join(__dirname, '/public')))
app.use('/subdir/innerdir',express.static(path.join(__dirname, '/public')))

app.use('/subdir',require('./Routes/subdir'))
app.use('/subdir/innerdir',require('./Routes/innerdir'))

app.use('/',require('./Routes/root'))

app.use(errorHandler)

app.listen( PORT , () =>

```

```
{  
  console.log('Server is running on',PORT)  
})
```

API

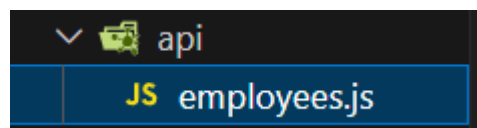
Step 1: Create Data json

Create a data Folder and a file employees.json

data>employees.json



Step 2: Create a api folder then create a file employees.js



api>employees.js:

```
const express = require('express')  
const path = require('path')  
const router = express.Router()  
  
data = {}  
data.employees = require('../data/employees.json')  
  
router.route('/')  
  .get((req,res)=>  
  {  
    res.json(data.employees)  
  })  
  .post((req,res)=>  
  {  
    res.json(  
      {  
        "firstname" : req.body.firstname,  
        "lastname" : req.body.lastname
```

```

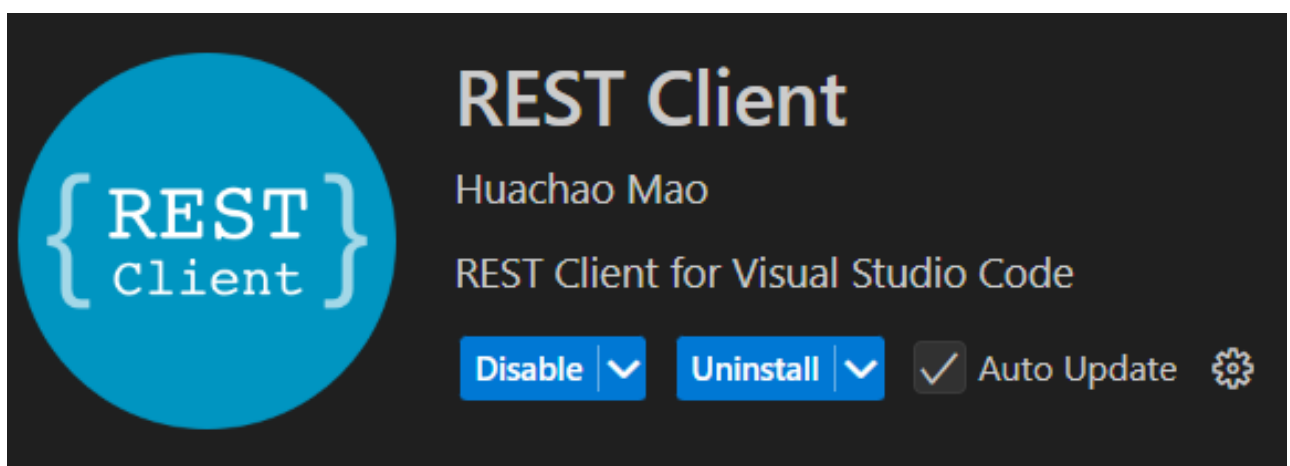
    }
  )
})
.put((req,res)=>
{
  res.json(
    {
      "firstname" : req.body.firstname,
      "lastname" : req.body.lastname
    }
  )
})
.delete((req,res)=>
{
  res.json({"id":req.body.id})
})

//id
router.route('/:id')
.get((req,res)=>
{
  res.json(
    {
      "id":req.params,
      "firstname" : req.query.firstname || 'NOT PROVIDED',
      "lastname" : req.query.lastname || 'NOT PROVIDED'
    }
  )
})

module.exports = router

```

Step 3: Install a extension called 'REST Client' in vs code to act as a frontend giving a request



Step 4: Add the highlighted line in web_server.js

```
const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents,logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')
//Custom Middleware
app.use(logger)
//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500','http://localhost:3500']
const corsOptions = {
  origin : (origin,callback) =>
  {
    if(whitelist.includes(origin) || !origin){
      callback(null,true)
    }
    else{
      callback(new Error('Blocked Due to Cross Policy !'))
    }
  },
  optionsSuccessStatus : 200
}
app.use(cors(corsOptions))
app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname,'/public')))

app.use('/employees',require('./api/employees'))

app.use('/subdir',express.static(path.join(__dirname,'/public')))
app.use('/subdir/innerdir',express.static(path.join(__dirname,'/public')))

app.use('/subdir',require('./Routes/subdir'))
app.use('/subdir/innerdir',require('./Routes/innerdir'))

app.use('/',require('./Routes/root'))

app.use(errorHandler)

app.listen( PORT , () =>
{
  console.log('Server is running on',PORT)
})
```

Step 5: create a file globally called 'route.rest'



When tap on send request the Results will be ::

GET :

A screenshot of the route.rest application interface. The left pane shows a REST client with a GET request to 'http://localhost:3500/employees'. A red arrow points to the 'Send Request' button. The right pane shows the response, which is a 200 OK status with a JSON array of two employee objects. The JSON is expanded to show the details of each employee.

```
1 GET http://localhost:3500/employees
2
3
4
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Vary: Origin
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 97
6 ETag: W/"61-Z3oPFAocUSPD+V4oEMSq1wOMGc"
7 Date: Tue, 18 Mar 2025 00:57:08 GMT
8 Connection: close
9
10 [
11   {
12     "id": 1,
13     "firstname": "Prasanth",
14     "lastname": "RS"
15   },
16   {
17     "id": 2,
18     "firstname": "Giridhar",
19     "lastname": "RS"
20   }
21 ]
```

GET + id :

A screenshot of the route.rest application interface. The left pane shows a GET request to 'http://localhost:3500/employees/1'. The right pane shows the response, which is a 200 OK status with a JSON object containing employee details for ID 1. The JSON is expanded to show the details.

```
1 GET http://localhost:3500/employees/1
2
3
4
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Vary: Origin
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 70
6 ETag: W/"46-qTQnGG21iSfgZHNaHZX15CrhA/E"
7 Date: Tue, 18 Mar 2025 01:01:28 GMT
8 Connection: close
9
10 {
11   "id": {
12     "id": "1"
13   },
14   "firstname": "NOT PROVIDED",
15   "lastname": "NOT PROVIDED"
16 }
```

POST:

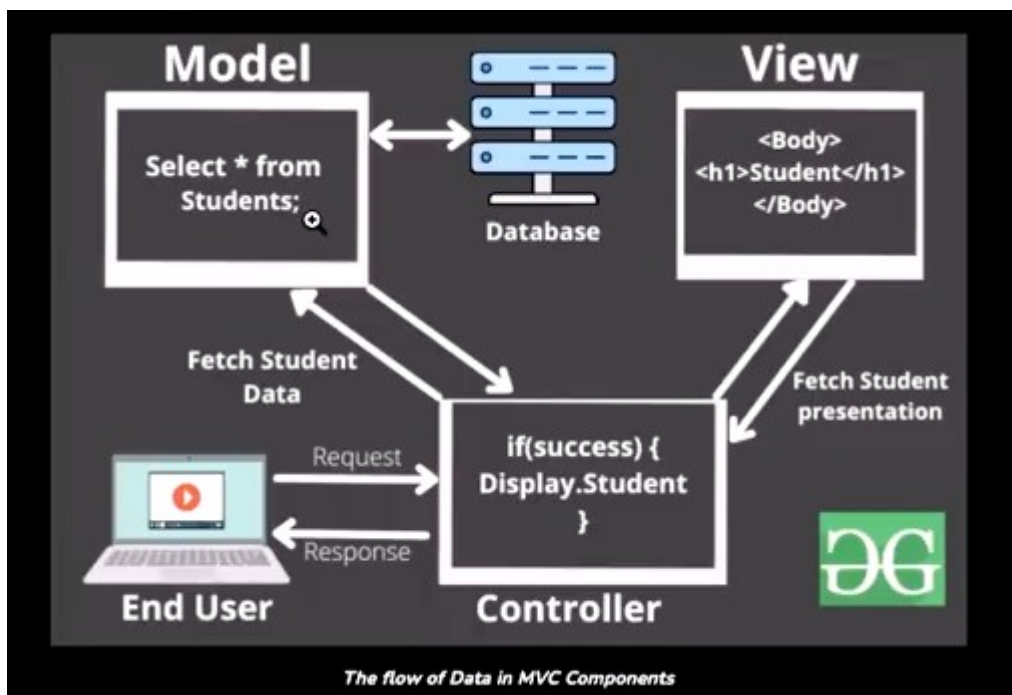
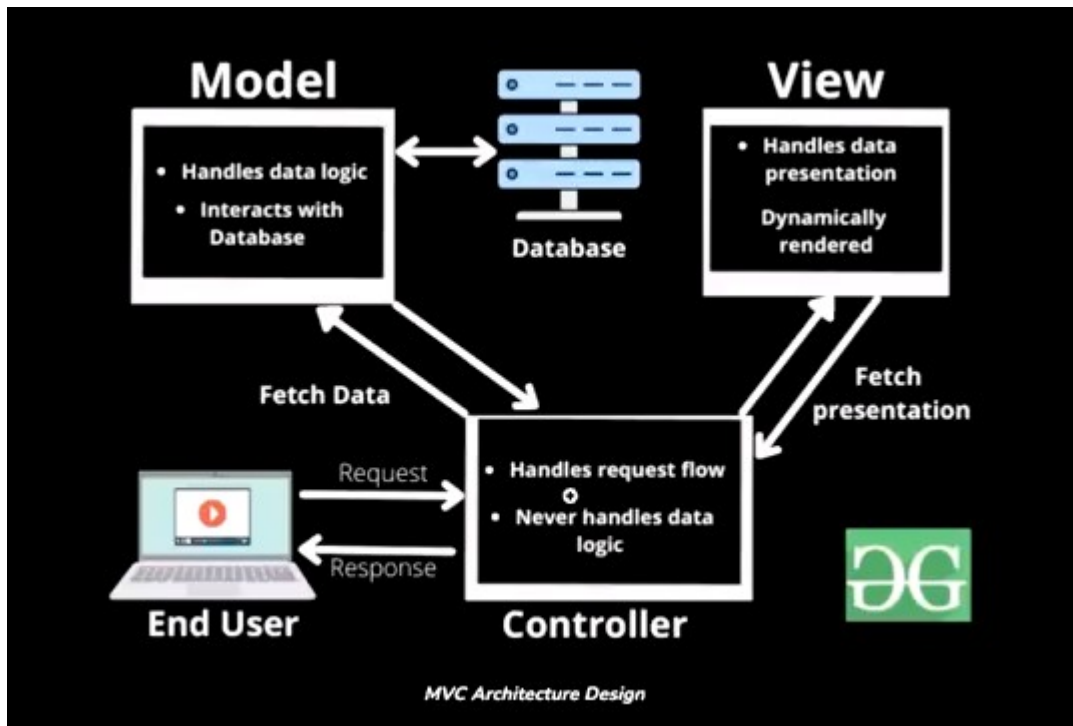
A screenshot of the route.rest application interface. The left pane shows a POST request to 'http://localhost:3500/employees' with a JSON body. The right pane shows the response, which is a 200 OK status with a JSON object containing employee details. The JSON is expanded to show the details.

```
1 POST http://localhost:3500/employees
2 Content-Type: application/json
3 {
4   "firstname": "Giridhar",
5   "lastname": "RS"
6 }
7
8
9
10
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Vary: Origin
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 40
6 ETag: W/"28-NJqZJEqiW5yPQN/IMsKkeEVmoxI"
7 Date: Tue, 18 Mar 2025 01:02:22 GMT
8 Connection: close
9
10 {
11   "firstname": "Giridhar",
12   "lastname": "RS"
13 }
```

MVC:

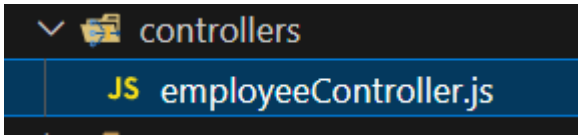
Files and folder apm allocation epadi venaalum pannalaam in Express.js , Adha confusion ah sari panna than oru c procedure than MVC.



Status code	Meaning
200 OK	Request was successful.
301 Moved Permanently	For SEO purposes when a page has been moved and all link equity should be passed through.
401 Unauthorized	Server requires authentication.
403 Forbidden	Client authenticated but does not have permissions to view resource.
404 Not Found	Page not found because no search results or may be out of stock.
500 Internal Server Error	Server side error. Usually due to bugs and exceptions thrown on the server side code.
503 Server Unavailable	Server side error. Usually due to a platform hosting, overload and maintenance issue.

Impotent :

Step 1 : Create a folder called controllers and create a file for controlling



employeeController.js:

```
const data = {
  employees : require('../model/employees.json'),
  setEmployees : function(data) {this.employees = data}
}

const getAllEmployees = (req,res)=>
{
  res.json(data.employees)
}

const createNewEmployee = (req,res)=>
{
  const newEmployee =
  {
    id : data.employees?.length? data.employees[data.employees.length
-1].id + 1 : 1,
    firstname : req.body.firstname,
    lastname : req.body.lastname
  }

  if(!newEmployee.firstname || !newEmployee.lastname)
  {
```

```

        res.status(400).json({"message":"Provide both firstname and
        Lastname"})
    }

    data.setEmployees([...data.employees,newEmployee])
    res.status(201).json(data.employees)
}

const updateEmployee = (req,res)=>
{
    const employee = data.employees.find(emp => emp.id ===
    parseInt(req.body.id))

    if(!employee)
    {
        res.status(400).json({"message":`Employee with id ${req.body.id}
is not found!`})
    }

    if(req.body.firstname) employee.firstname = req.body.firstname
    if(req.body.lastname) employee.lastname = req.body.lastname

    const filteredArray = data.employees.filter(emp => emp.id !==
    parseInt(req.body.id))

    const unsortedArray = [...filteredArray,employee]
    data.setEmployees(unsortedArray.sort((a,b) => a.id > b.id ? 1 : a.id <
    b.id ? -1:0))
    res.json(data.employees)
}

const deleteEmployee = (req,res)=>
{
    const employee = data.employees.find(emp => emp.id ===
    parseInt(req.body.id))

    if(!employee)
    {
        res.status(400).json({"message":`Employee with id ${req.body.id}
is not found!`})
    }

    const filteredArray = data.employees.filter(emp => emp.id !==
    parseInt(req.body.id))

    data.setEmployees([...filteredArray])

    res.status(201).json(data.employees)
}

```

```

    }

const getEmployee = (req,res)=>
{
    const employee = data.employees.find(emp => emp.id ===
parseInt(req.body.id))

    if(!employee)
    {
        res.status(400).json({"message":`Employee with id ${req.body.id}
is not found!`})
    }

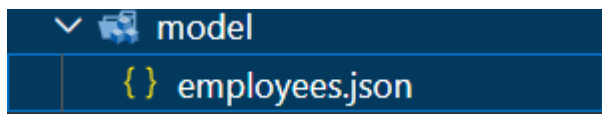
    res.status(201).json(employee)

}

module.exports =
{getAllEmployees,createNewEmployee,updateEmployee,deleteEmployee,getEmployee}

```

Step 2: Create a model folder for data storage (JSON)



employees.json:

```

[
  {
    "id" :1,
    "firstname" : "Prasanth",
    "lastname" : "RS"
  },
  {
    "id" :2,
    "firstname" : "Giridhar",
    "lastname" : "RS"
  }
]

```

corsOptions.js:

```
//Cors access giving and declining
const whitelist = ['https://www.goo.com',
'http://127.0.0.1:5500', 'http://localhost:3500']
const corsOptions = {
  origin : (origin, callback) =>
  {
    if(whitelist.includes(origin) || !origin)
    {
      callback(null, true)
    }
    else
    {
      callback(new Error('Blocked Due to Cross Policy !'))
    }
  },
  optionsSuccessStatus : 200
}
```

Step 3: Alter the web_server.js based on the files and directories

```
//=====MVC=====
const express = require('express')
const app = express()
const path = require('path')
const PORT = process.env.PORT || 3500
const {logEvents, logger} = require('./middleware/logEvents')
const cors = require('cors')
const errorHandler = require('./middleware/errorHandler')

const corsOptions = require('./corsOptions')

//Custom Middleware
app.use(logger)

app.use(cors(corsOptions))

app.use(express.urlencoded({extended:false})) // built-in middleware
app.use(express.json())
app.use(express.static(path.join(__dirname, '/public')))

app.use('/employees', require('./api/employees'))

app.use('/subdir', express.static(path.join(__dirname, '/public')))
app.use('/subdir/innerdir', express.static(path.join(__dirname, '/public')))
```

```
app.use('/subdir',require('./Routes/subdir'))
app.use('/subdir/innerdir',require('./Routes/innerdir'))

app.use('/',require('./Routes/root'))

app.use(errorHandler)

app.listen( PORT , () =>
{
  console.log('Server is running on',PORT)
})
```